# A. Sum of Numbers I

| **Time limit** | 1 second |
|---|---|
| **Memory limit** | 256 megabytes |

Given integers $n$ and $m$, compute their sum.

## Input format

The single line contains integers $n$ and $m$ ($1 \le n, m \le 1000$).

## Output format

Print a single integer — the sum $n + m$.

## Samples

| # | stdin | stdout |
|---|---|---|
| 1 | 3 4 | 7 |

## Scoring

| Subgroup | Limits | Score | Required subgroups |
|---|---|---|---|
| 0 | Sample tests | 0 | — |
| 1 | $(1 \le n, m \le 1000)$ | 100 | 0 |

# B. Sum of Numbers II

| **Time limit** | 1 second |
|---|---|
| **Memory limit** | 256 megabytes |

Given integers $n$ and $m$, compute their sum.

## Input format

The single line contains integers $n$ and $m$ ($1 \le n, m \le 1000$).

## Output format

Print a single integer — the sum $n + m$.

## Samples

| # | stdin | stdout |
|---|---|---|
| 1 | 3 4 | 7 |

## Scoring

| Subgroup | Limits | Score | Required subgroups |
|---|---|---|---|
| 0 | Sample tests | 0 | — |
| 1 | $(1 \le n, m \le 500)$ | 50 | 0 |
| 2 | $(1 \le n, m \le 1000)$ | 50 | — |

# C. Sum of Numbers III

| Time limit | 1 second |
|---|---|
| Memory limit | 256 megabytes |

Given integers $n$ and $m$, compute their sum.

## Input format

The single line contains integers $n$ and $m$ ($1 \le n, m \le 1000$).

## Output format

Print a single integer — the sum $n + m$.

## Samples

| # | stdin | stdout |
|---|---|---|
| 1 | 3 4 | 7 |

## Scoring

| Subgroup | Limits | Score | Required subgroups |
|---|---|---|---|
| 0 | Sample tests | 0 | — |
| 1 | $(1 \le n, m \le 1000)$ | 50 | 179 |

# D. Sum of Numbers IV

| Time limit | 1 second |
|---|---|
| Memory limit | 256 megabytes |

**This is a problem with subgroup merging.**
Given integers $n$ and $m$, compute their sum.

## Input format

The single line contains integers $n$ and $m$ ($1 \le n, m \le 1000$).

## Output format

Print a single integer — the sum $n + m$.

## Samples

| # | stdin | stdout |
|---|---|---|
| 1 | 3 4 | 7 |

## Scoring

| Subgroup | Limits | Score | Required subgroups |
|---|---|---|---|
| 0 | Sample tests | 0 | — |
| 1 | $(1 \le n, m \le 500)$ | 50 | 0 |
| 2 | $(1 \le n, m \le 1000)$ | 50 | — |

# E. Sum of Numbers V

| Time limit | 1 second |
|---|---|
| Memory limit | 256 megabytes |

**This is a problem with per-test scoring.**
Given integers $n$ and $m$, compute their sum.

## Input format
The single line contains integers $n$ and $m$ ($1 \le n, m \le 1000$).

## Output format
Print a single integer — the sum $n + m$.

## Samples

| # | stdin | stdout |
|---|---|---|
| 1 | 3 4 | 7 |

## Scoring
**This problem has two tests in addition to the sample tests; each of them is scored independently and is worth 50 points.**

# F. Guess the Number

| Time limit | 1 second |
|---|---|
| Memory limit | 256 megabytes |

**This is an interactive problem.**

An integer from 1 to $t$ is chosen, where $t$ is a positive integer reported to the program. At most $k$ queries of the form `? x` may be made ($k$ is a hidden number):

- if $x$ equals the chosen number, the interactor replies `=`, and the program must terminate immediately;
- if the chosen number is less than $x$, the interactor replies `<`;
- otherwise the interactor replies `>`.

## Interactive protocol

First, the interactor reports a positive integer $t$ $(1 \le t \le 10000)$ — the upper bound on the chosen number.

Then the program may make at most $t$ queries of the form `? x`, where $x$ is an integer from 1 to $t$. To each query the interactor replies with one of the symbols `<`, `>` or `=`.

After receiving the reply `=`, the program must terminate immediately.

## Samples

| # | Interactor | Solution |
|---|---|---|
| 1 | 10<br>><br><<br>= | <br>? 5<br>? 8<br>? 7 |

## Scoring

| Subgroup | Limits | Score | Required subgroups |
|---|---|---|---|
| 0 | Sample tests | 0 | — |
| 1 | $(1 \le t \le 10000)$ | 100 | 0 |

# G. Add 1

| Time limit | 1 second |
|---|---|
| Memory limit | 256 megabytes |

**This is a run-twice problem.**
The program is run twice. In the first run it receives an integer $n$ and must print $n + 1$. In the second run it receives the value printed by the first run and must recover the original number $n$. The second run receives no other information about $n$.

## The first run
The single line contains an integer $n$ ($1 \le n \le 1000$). Print $n + 1$.

## The second run
The single line contains an integer $v$ ($2 \le v \le 1001$) — the result of the first run. Print $v - 1$.

## Samples

| # | Run | stdin | stdout |
|---|---|---|---|
| 1 | The first run | `0`<br>`10` | `11` |
| 1 | The second run | `1`<br>`11` | `10` |

## Comments
The example shows both runs separately.

## Scoring

| Subgroup | Limits | Score | Required subgroups |
|---|---|---|---|
| 0 | Sample tests | 0 | — |
| 1 | ($1 \le n \le 1000$) | 100 | 0 |